

**LAPORAN PRAKTIKUM
APLIKASI BERBASIS PLATFORM**

**MODUL 1–3
GIT, FLUTTER & DART**



Disusun oleh:
D'SHARLENDITA FEBIANDA AURELIA
2311102069
S1 IF-11-04

Dosen Pengampu:
Cahyo Prihantoro, S.Kom., M.Eng

**PROGRAM STUDI S1 INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO**

2026

DASAR TEORI

1. Dasar Teori Git

Git merupakan salah satu sistem pengontrol versi (*Version Control System / VCS*) yang digunakan dalam pengembangan perangkat lunak. Git diciptakan oleh Linus Torvalds dan berfungsi untuk mencatat setiap perubahan yang terjadi pada file dalam suatu proyek, baik yang dikerjakan secara individu maupun dalam tim. Dengan adanya Git, setiap perubahan dapat dilacak, sehingga memudahkan pengembang dalam mengelola versi kode serta melakukan kolaborasi dengan lebih efektif.

Git termasuk dalam kategori *distributed version control system*, yang berarti setiap pengguna memiliki salinan lengkap dari repository beserta riwayat perubahannya. Hal ini berbeda dengan sistem terpusat yang hanya menyimpan data pada satu server. Dengan sistem terdistribusi, Git menjadi lebih fleksibel, aman, dan tidak bergantung pada satu titik pusat, sehingga risiko kehilangan data dapat diminimalkan.

Secara umum, Git bekerja dengan cara menyimpan snapshot dari perubahan file, bukan hanya menyimpan perbedaan antar file. Setiap perubahan yang dilakukan akan disimpan dalam bentuk commit yang memiliki identitas unik. Commit ini memungkinkan pengguna untuk kembali ke versi sebelumnya apabila terjadi kesalahan, serta membandingkan perubahan antar versi dengan mudah.

Selain itu, Git juga memiliki konsep *branching* yang memungkinkan pengembang membuat cabang pengembangan secara terpisah tanpa mengganggu kode utama. Fitur ini sangat penting dalam pengembangan perangkat lunak karena memungkinkan beberapa fitur dikembangkan secara paralel. Setelah selesai, perubahan dari branch dapat digabungkan kembali (*merge*) ke dalam branch utama.

Git biasanya digunakan bersamaan dengan platform seperti GitHub, GitLab, atau Bitbucket untuk mempermudah kolaborasi secara online. Platform tersebut menyediakan fitur tambahan seperti manajemen repository, tracking issue, serta integrasi dengan berbagai tools pengembangan lainnya.

2. Dasar Teori Flutter

Flutter merupakan framework yang dikembangkan oleh Google untuk membangun aplikasi berbasis mobile, web, dan desktop dengan menggunakan satu basis kode (*single codebase*). Flutter ditulis menggunakan bahasa pemrograman Dart dan menggunakan *rendering engine* sendiri yaitu Skia untuk menampilkan antarmuka pengguna (*user interface*).

Flutter bekerja dengan memanfaatkan Dart Virtual Machine (VM) yang mendukung kompilasi *Just-In-Time* (JIT). Hal ini memungkinkan fitur *hot reload*, yaitu kemampuan untuk melihat perubahan kode secara langsung tanpa perlu menjalankan ulang aplikasi. Fitur ini sangat membantu dalam proses pengembangan karena dapat mempercepat debugging dan iterasi desain aplikasi.

Salah satu konsep utama dalam Flutter adalah penggunaan widget. Semua komponen dalam aplikasi Flutter, baik itu tampilan maupun struktur, direpresentasikan sebagai widget. Widget dapat bersifat *stateless* (tidak memiliki perubahan state) maupun

stateful (memiliki perubahan state). Konsep ini memungkinkan pengelolaan antarmuka yang lebih modular dan terstruktur.

Selain itu, Flutter memiliki arsitektur yang mendukung pemisahan antara logika bisnis dan tampilan, salah satunya melalui pendekatan BLoC (*Business Logic Component*). Pendekatan ini membantu dalam mengelola state aplikasi dengan lebih baik, sehingga kode menjadi lebih mudah dipelihara, scalable, dan dapat diuji.

3. Dasar Teori Dart

Dart merupakan bahasa pemrograman yang dikembangkan oleh Google dan digunakan sebagai bahasa utama dalam pengembangan aplikasi menggunakan Flutter. Dart dirancang untuk mendukung pengembangan aplikasi modern yang cepat, efisien, dan mudah dipelajari, terutama bagi pengembang yang sudah familiar dengan bahasa seperti Java atau JavaScript.

Dart memiliki sintaks yang sederhana dan bersifat *object-oriented*, sehingga memungkinkan penggunaan konsep seperti class, object, inheritance, dan encapsulation. Selain itu, Dart juga mendukung pemrograman asinkron melalui fitur seperti *Future* dan *async/await*, yang sangat penting dalam pengembangan aplikasi yang membutuhkan pengolahan data secara bersamaan, seperti pengambilan data dari internet.

Dalam Dart, terdapat beberapa konsep dasar yang penting, seperti variabel, struktur kontrol (*if-else*, *switch*), perulangan (*looping*), fungsi, dan struktur data seperti list. Konsep-konsep ini menjadi dasar dalam membangun logika program dalam aplikasi Flutter.

Dart juga memiliki kemampuan untuk dikompilasi menjadi kode native, sehingga aplikasi yang dibuat dapat berjalan dengan performa tinggi di berbagai platform. Dalam Flutter, Dart sudah terintegrasi langsung dalam Flutter SDK, sehingga tidak perlu melakukan instalasi terpisah.

Sumber Referensi

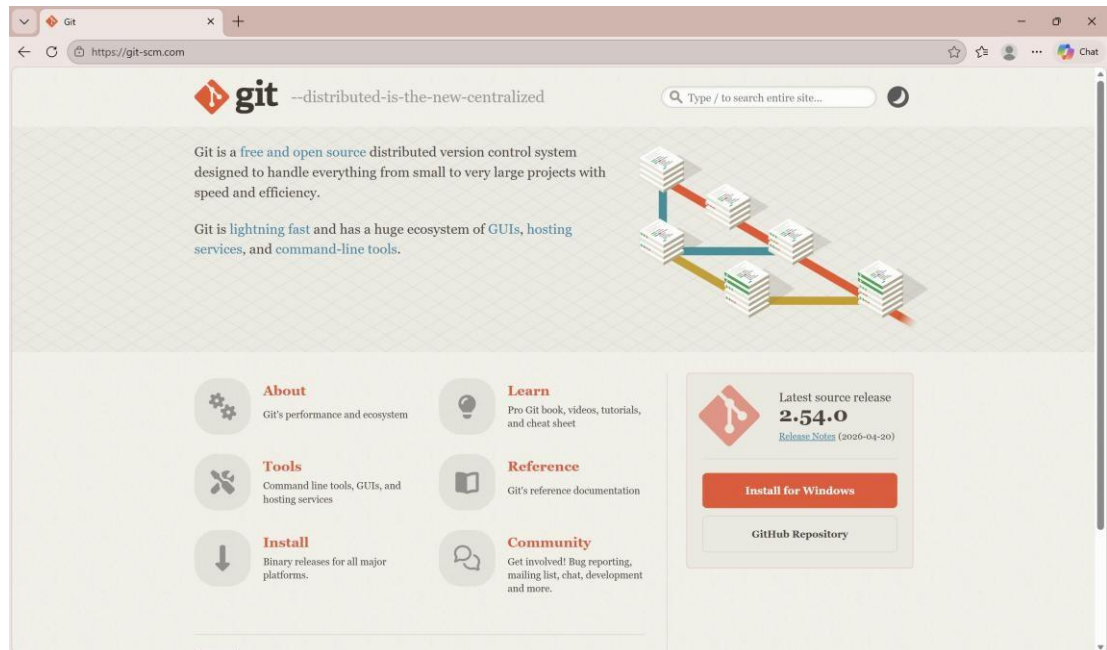
1. Modul Praktikum Pemrograman Perangkat Bergerak, Telkom University Purwokerto (2024)
2. Chacon, S., & Straub, B. (2014). *Pro Git* (2nd Edition). Apress.
3. Git Documentation. (2024). *Git Official Documentation*. <https://git-scm.com/docs>
4. Loeliger, J., & McCullough, M. (2012). *Version Control with Git*. O'Reilly Media
5. Google. (2024). *Flutter Documentation*. <https://flutter.dev/docs>
6. Google. (2024). *Dart Documentation*. <https://dart.dev/guides>
7. Tilkov, S., & Vinoski, S. (2010). *Node.js: Using JavaScript to Build High-Performance Network Programs* (untuk perbandingan konsep async)

GUIDED MODUL 1 RUNNING MODUL

Pada modul ini dilakukan proses instalasi dan konfigurasi tools yang dibutuhkan untuk pengembangan aplikasi Flutter.

1. Instalasi Git

- a. Mengunduh Git melalui website resmi (<https://git-scm.com>)



- b. Menjalankan installer dan mengikuti proses instalasi hingga selesai
c. Melakukan konfigurasi awal:

```
git config --global user.name "Nama"  
git config --global user.email "email@contoh.com"
```

- d. Memastikan instalasi berhasil dengan perintah:

```
git --version
```

```
Command Prompt
Microsoft Windows [Version 10.0.26200.8246]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Dita>git config --list
diff.astextplain.textconv=astextplain
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
http.sslbackend=schannel
core.autocrlf=true
core.fscache=true
core.symlinks=false
pull.rebase=false
credential.helper=manager
credential.https://dev.azure.com.usehttppath=true
init.defaultbranch=master
user.name=Dita
user.email=2311102069@ittelkom-pwt.ac.id
core.editor="C:\Users\Dita\AppData\Local\Programs\Microsoft VS Code\bin\code" --wait

C:\Users\Dita>
```

Berdasarkan hasil pengecekan menggunakan perintah `git config --list`, Git pada perangkat telah berhasil dikonfigurasi. Username yang digunakan adalah *Dita* dan email yang terdaftar adalah *2311102069@ittelkom-pwt.ac.id*. Selain itu, Git juga telah terintegrasi dengan Visual Studio Code sebagai editor default, sehingga memudahkan dalam proses pengembangan dan manajemen kode.

```
Command Prompt
Microsoft Windows [Version 10.0.26200.8246]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Dita>git --version
git version 2.53.0.windows.1

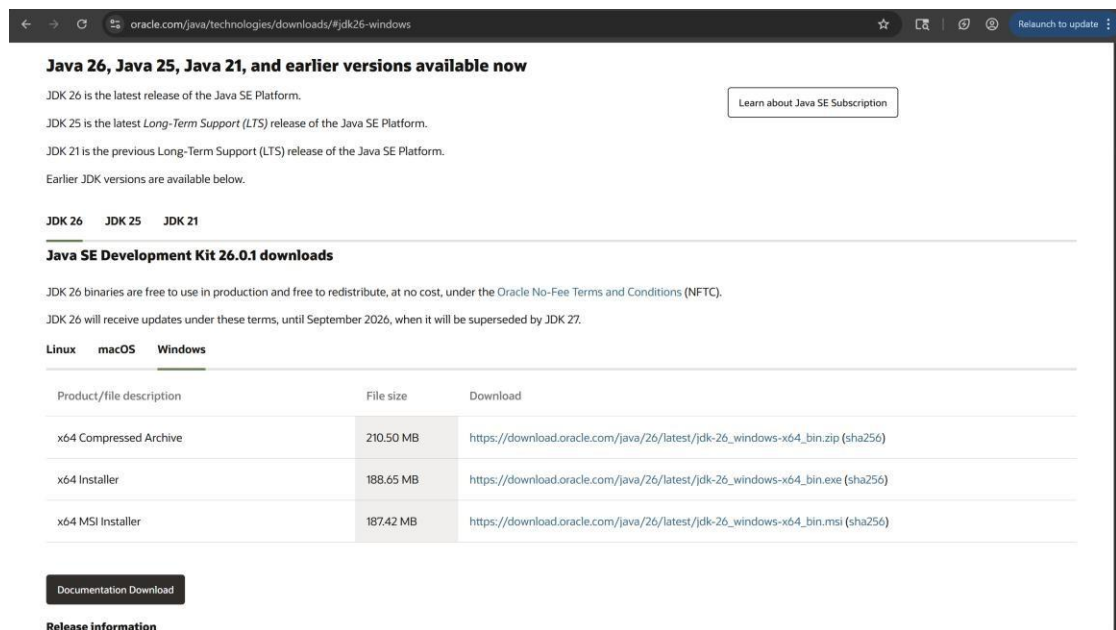
C:\Users\Dita>
```

Selanjutnya, dilakukan pengecekan versi Git menggunakan perintah `git --version`, yang menunjukkan bahwa Git telah terinstal dengan versi 2.53.0, sehingga dapat dipastikan Git siap digunakan dalam praktikum.

2. Instalasi JDK

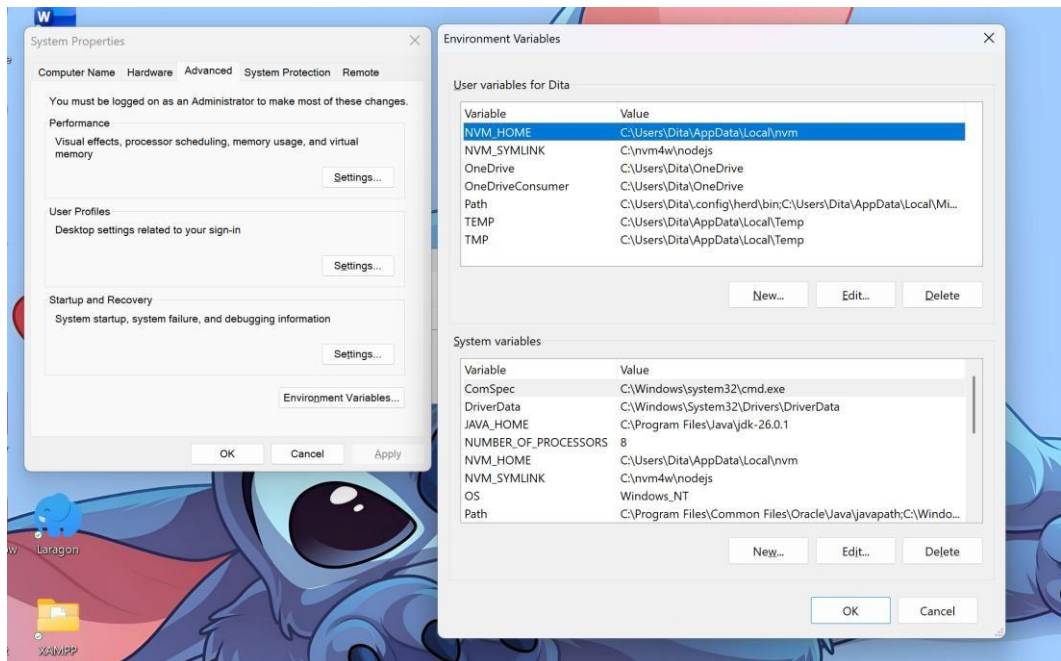
- a. Mengunduh JDK dari website resmi Oracle

<https://www.oracle.com/java/technologies/javase-downloads.html>

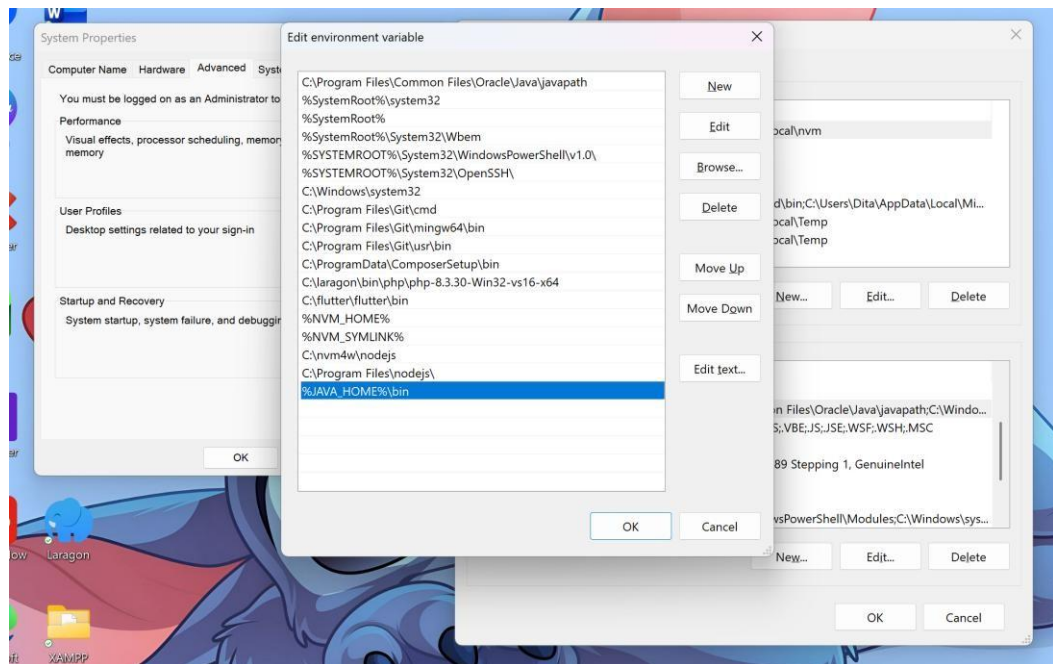


- b. Setelah file berhasil diunduh, dilakukan proses instalasi JDK sesuai dengan sistem operasi yang digunakan.
- c. Selanjutnya dilakukan konfigurasi *environment variable* dengan menambahkan variabel `JAVA_HOME` yang mengarah ke direktori instalasi JDK, serta menambahkan Path ke folder bin pada JDK.

- `JAVA_HOME`



- Path ke folder JDK



```
Microsoft Windows [Version 10.0.26200.8246]
(c) Microsoft Corporation. All rights reserved.

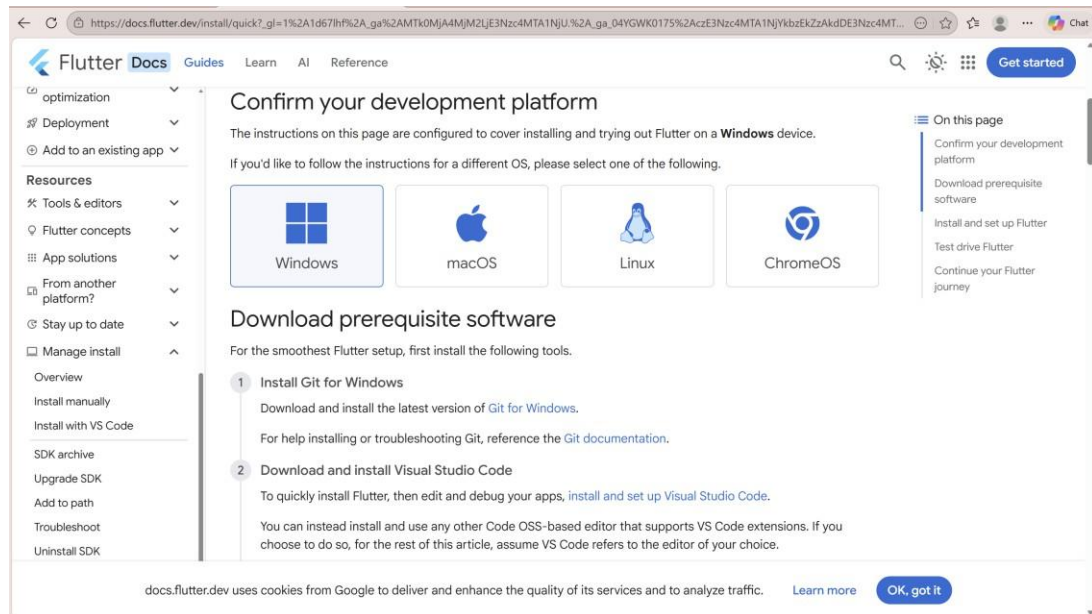
C:\Users\Dita>java --version
java 26.0.1 2026-04-21
Java(TM) SE Runtime Environment (build 26.0.1+8-34)
Java HotSpot(TM) 64-Bit Server VM (build 26.0.1+8-34, mixed mode, sharing)

C:\Users\Dita>
```

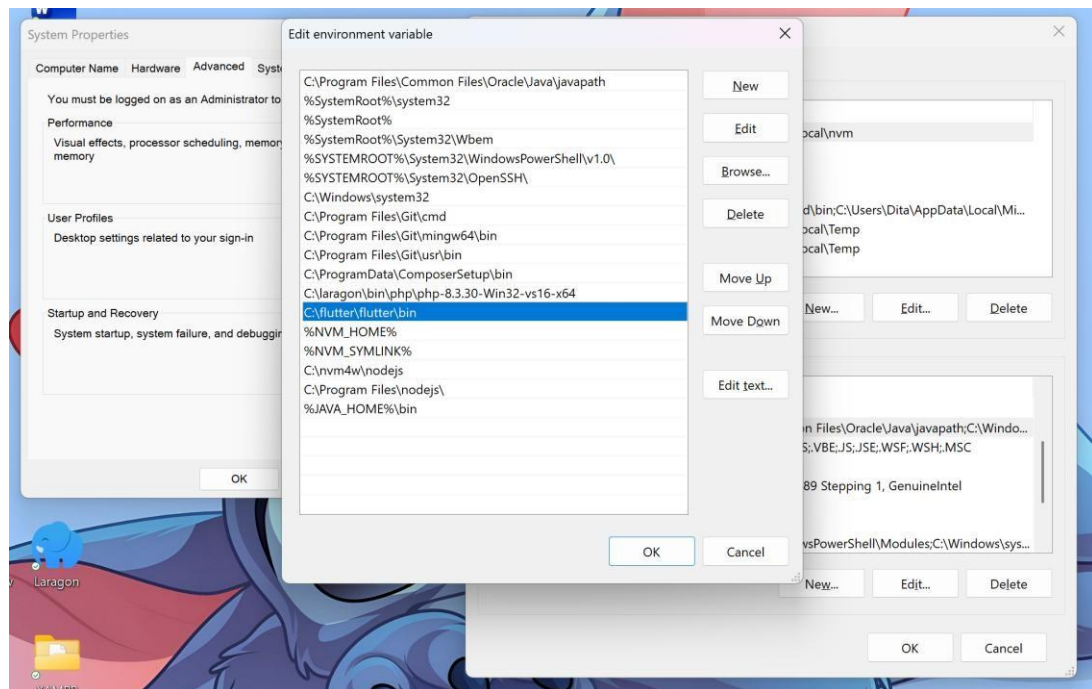
Berdasarkan hasil pengecekan menggunakan perintah `java --version`, JDK telah berhasil terinstal dengan versi 26.0.1.

3. Instalasi Flutter SDK

- a. Mengunduh Flutter SDK dari <https://flutter.dev>



- b. File yang telah diunduh kemudian diekstrak ke direktori tertentu (misalnya C:\flutter).
- c. Selanjutnya dilakukan konfigurasi *environment variable* dengan menambahkan path flutter\bin.



- d. Setelah itu, dilakukan pengecekan dengan menjalankan perintah:

```
flutter doctor
```

- e. Memastikan semua dependensi sudah terpenuhi


```
Command Prompt - flutter d X + v
Microsoft Windows [Version 10.0.26200.8246]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Dita>flutter doctor

A new version of Flutter is available!

To update to the latest version, run "flutter upgrade".

Doctor summary (to see all details, run flutter doctor -v):
[✓] Flutter (Channel stable, 3.41.7, on Microsoft Windows [Version 10.0.26200.8246], locale en-ID)
[✓] Windows Version (11 Home Single Language 64-bit, 25H2, 2009)
[✓] Android toolchain - develop for Android devices (Android SDK version 36.1.0)
[✓] Chrome - develop for the web
[✗] Visual Studio - develop Windows apps
    X Visual Studio not installed; this is necessary to develop Windows apps.
      Download at https://visualstudio.microsoft.com/downloads/.
      Please install the "Desktop development with C++" workload, including all of its default components
[✓] Connected device (3 available)
[✓] Network resources

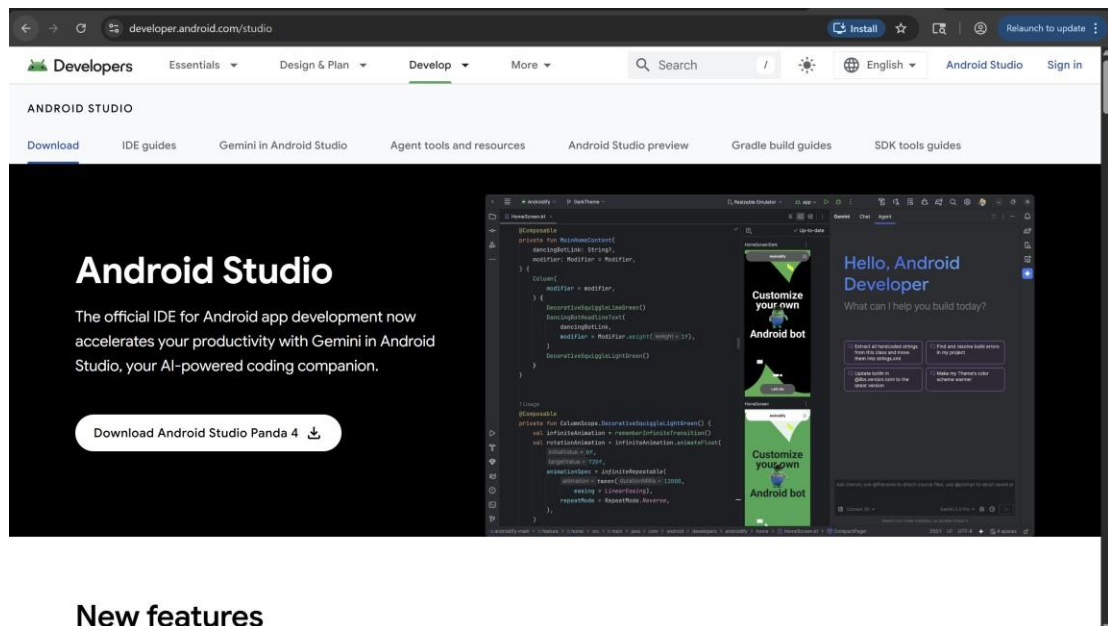
! Doctor found issues in 1 category.

C:\Users\Dita>
```

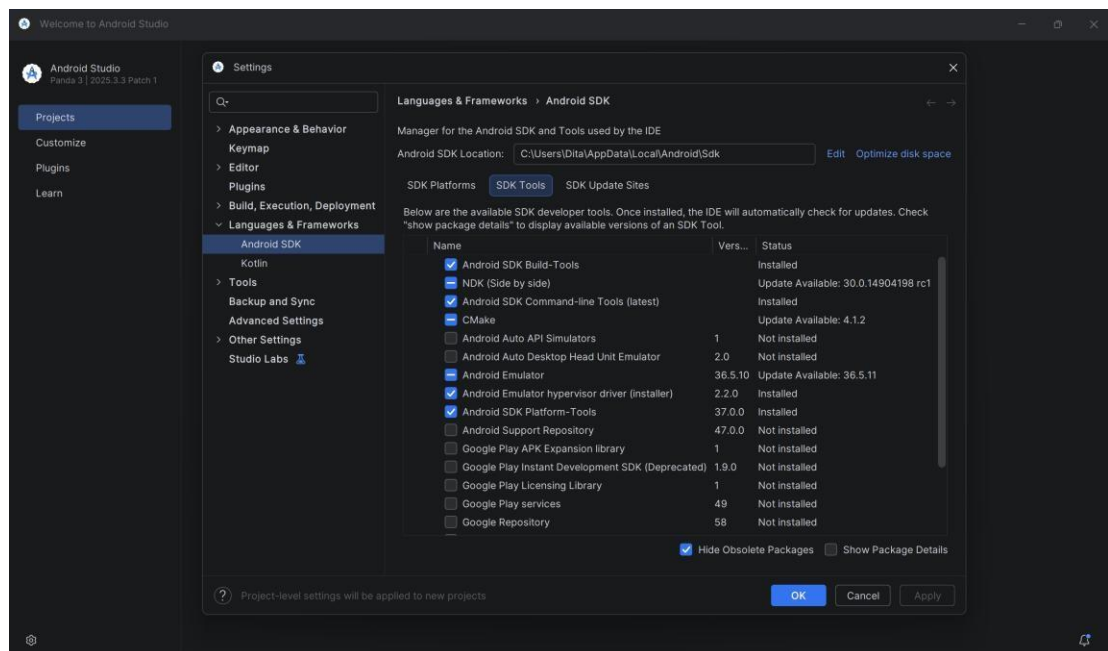
Berdasarkan hasil pengecekan, sebagian besar komponen seperti Flutter SDK, Android toolchain, dan perangkat telah terdeteksi dengan baik. Terdapat satu peringatan terkait Visual Studio yang belum terinstal, namun hal tersebut tidak mempengaruhi proses pengembangan aplikasi Flutter (Android).

4. Instalasi Android Studio

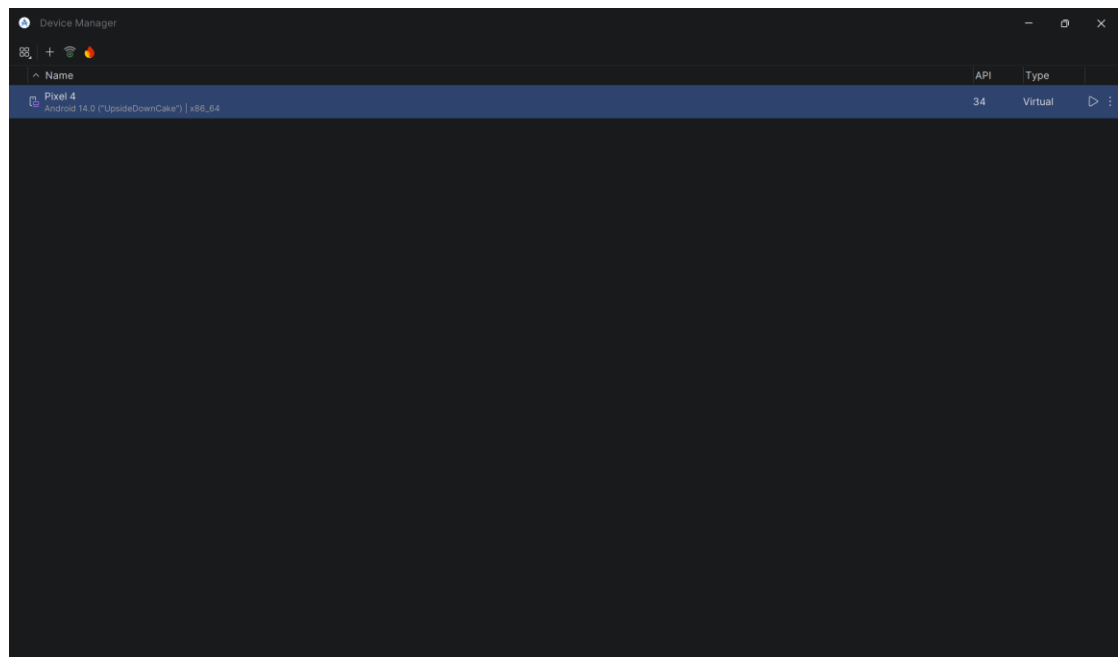
- a. Mengunduh Android Studio dari <https://developer.android.com/studio>



- b. File installer dijalankan, kemudian mengikuti proses instalasi dengan menekan tombol Next hingga selesai.
- c. Android SDK dan emulator diinstal melalui Android Studio dengan memilih menu More Actions, kemudian klik SDK Manager untuk menginstal komponen yang diperlukan pada bagian SDK Tools.



Berdasarkan hasil instalasi, Android Studio berhasil dijalankan dan Android SDK telah terpasang dengan baik, ditunjukkan melalui menu SDK Manager. Beberapa komponen penting seperti Build-Tools, Platform-Tools, dan Emulator sudah terinstal.



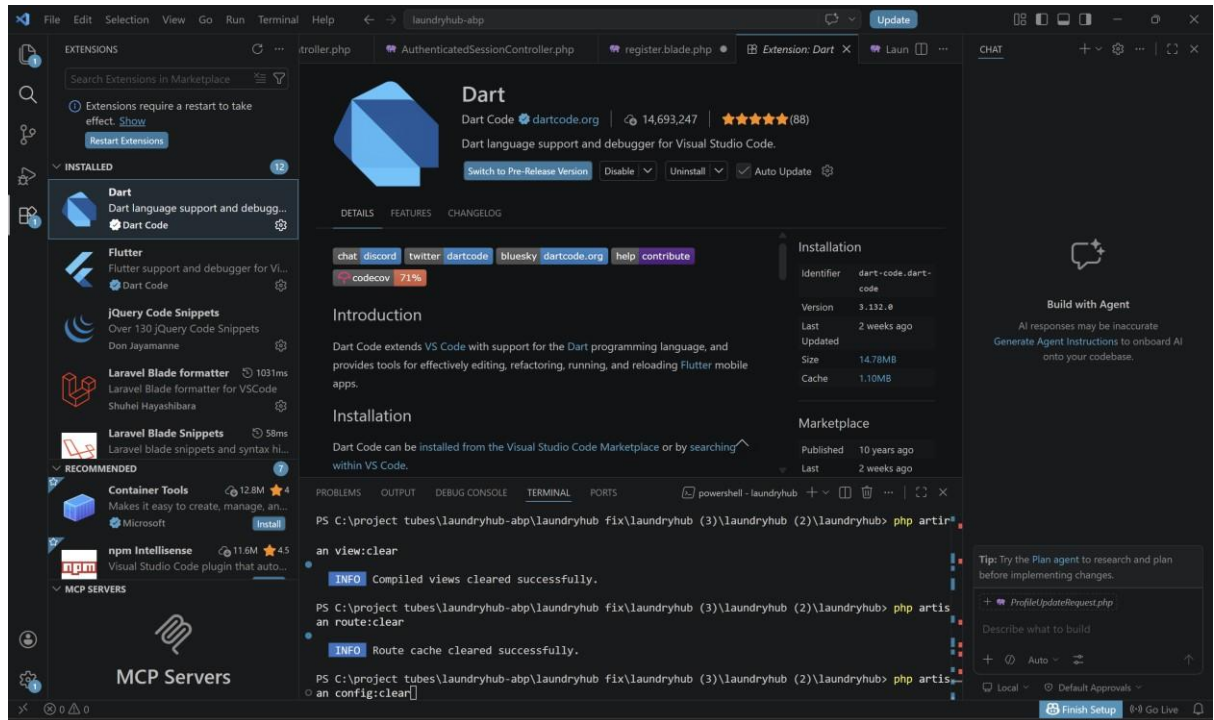
Selain itu, melalui *Virtual Device Manager* telah dibuat emulator Pixel 4 dengan sistem operasi Android 14 (API 34).

5. Instalasi Extension di Visual Studio Code

Extension yang diinstal pada Visual Studio Code yaitu:

- Dart
- Flutter

Extension Dart digunakan untuk menjalankan bahasa pemrograman Dart, sedangkan extension Flutter digunakan untuk mendukung pengembangan aplikasi Flutter di Visual Studio Code.



GUIDED

MODUL 2 PENGENALAN FLUTTER

Pada modul ini dilakukan pembuatan dan menjalankan project Flutter sederhana menggunakan Visual Studio Code. Project dijalankan dengan perintah flutter run hingga aplikasi muncul di emulator.

1. Membuat Project Flutter

Project dibuat menggunakan Visual Studio Code dengan memilih perintah Flutter: New Project, kemudian menentukan jenis aplikasi (Application), lokasi penyimpanan, dan nama project.

2. Menjalankan Project

Project dijalankan melalui terminal dengan perintah flutter run setelah emulator atau perangkat telah aktif.

3. Membuat Program Hello World

File main.dart dimodifikasi dengan menambahkan kode sederhana untuk menampilkan teks “Hello World”, kemudian aplikasi dijalankan kembali untuk melihat hasilnya.

```
// ignore_for_file: prefer_const_constructors

import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: "Modul 2",
      home: const MyHomePage(title: "Modul 2: Flutter"),
    );
  }
}

class MyHomePage extends StatefulWidget {
  const MyHomePage({Key? key, required this.title}) :
    super(key: key);

  final String title;

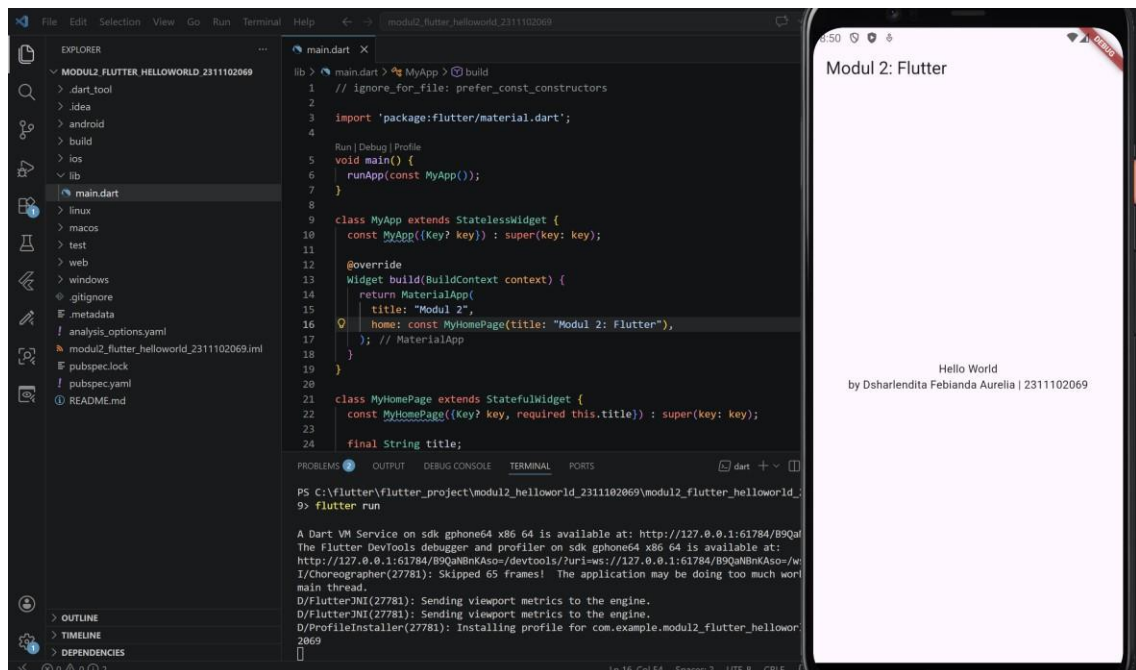
  @override
  State<MyHomePage> createState() => _MyHomePageState();
}

class _MyHomePageState extends State<MyHomePage> {
  @override
  Widget build(BuildContext context) {
```

```

return Scaffold(
  appBar: AppBar(
    title: Text(widget.title),
  ),
  body: Center(
    child: Column(
      mainAxisAlignment: MainAxisAlignment.center,
      children: [
        Text('Hello World'),
        Text('by Dsharlendita Febianda Aurelia |
2311102069'),
      ],
    ),
  ),
);
}
}

```



Program yang dibuat menampilkan teks “Hello World” dan identitas di layar. Hal ini menunjukkan bahwa aplikasi Flutter sudah berhasil berjalan dengan baik. Penggunaan widget seperti MaterialApp, Scaffold, AppBar, dan Text sudah sesuai dengan struktur dasar Flutter. Selain itu, penggunaan Column dan Center membuat tampilan teks berada di tengah dan tersusun rapi. Dari hasil yang ditampilkan di emulator, aplikasi dapat berjalan tanpa error.

GUIDED

MODUL 3 PENGENALAN DART

Pada modul ini dipelajari dasar-dasar bahasa Dart seperti variabel, percabangan, perulangan, list, dan fungsi.

1. Membuat Variabel
Membuat variabel dengan tipe data tertentu seperti String dan int, kemudian menampilkan nilainya menggunakan fungsi print().
2. Statement Control (Percabangan)
Menggunakan percabangan if-else untuk menentukan kondisi tertentu dan menghasilkan output sesuai dengan nilai yang diberikan.
3. Looping (Perulangan)
Menerapkan perulangan menggunakan for untuk menampilkan data secara berulang sesuai jumlah yang ditentukan.
4. List (Array)
Membuat dan mengakses data menggunakan struktur data List untuk menyimpan beberapa nilai dalam satu variabel.
5. Fungsi
Membuat fungsi sederhana untuk melakukan operasi tertentu, kemudian memanggil fungsi tersebut di dalam program utama.

```
void main() {  
  // 1. Variabel  
  String nama = "Dita";  
  int umur = 20;  
  
  print("Nama: $nama");  
  print("Umur: $umur");  
  
  // 2. Percabangan (if-else)  
  int nilai = 80;  
  
  if (nilai >= 75) {  
    print("Status: Lulus");  
  } else {  
    print("Status: Tidak Lulus");  
  }  
  
  // 3. Looping (Perulangan)  
  print("Perulangan:");  
  for (int i = 1; i <= 5; i++) {  
    print("Angka ke-$i");  
  }  
  
  // 4. List (Array)  
  List<String> buah = ["Apel", "Jeruk", "Mangga"];  
  print("Buah pertama: ${buah[0]}");  
  
  // 5. Fungsi
```



```

    int hasil = tambah(2, 3);
    print("Hasil penjumlahan: $hasil");
}

// Fungsi
int tambah(int a, int b) {
    return a + b;
}

```

The screenshot shows a VS Code editor with a Dart file named `main.dart`. The code defines a `tambah` function and a `main` function. The `main` function includes variable declarations, conditional logic, a loop, and a list. The terminal output shows the execution results.

```

1 void main() {
2   // 1. Variabel
3   String nama = "Dita";
4   int umur = 20;
5
6   print("Nama: $nama");
7   print("Umur: $umur");
8
9   // 2. Percabangan (if-else)
10  int nilai = 80;
11
12  if (nilai >= 75) {
13    print("Status: Lulus");
14  } else {
15    print("Status: Tidak Lulus");
16  }
17
18  // 3. Looping (Perulangan)
19  print("Perulangan:");
20  for (int i = 1; i <= 5; i++) {
21    print("Angka ke-$i");
22  }
23
24  // 4. List
25  List buah = ["Apel", "Jeruk", "Mangga", "Pisang"];
26  print("Buah pertama: ${buah.first}");
27
28  // 5. Hasil penjumlahan
29  int hasil = tambah(2, 3);
30  print("Hasil penjumlahan: $hasil");
31 }

```

```

PS C:\Users\Dita\.vscode\Modul3_Dart_S1IF1104_2311102069_D'sharlendra Febianda Aurelia> dart main.dart
Nama: Dita
Umur: 20
Status: Lulus
Perulangan:
Angka ke-1
Angka ke-2
Angka ke-3
Angka ke-4
Angka ke-5
Buah pertama: Apel
Hasil penjumlahan: 5

```

Variabel digunakan untuk menyimpan data seperti nama dan umur. Percabangan digunakan untuk menentukan kondisi berdasarkan nilai. Perulangan digunakan untuk menampilkan data secara berulang, sedangkan list digunakan untuk menyimpan beberapa data dalam satu variabel. Selain itu, dibuat fungsi sederhana untuk melakukan penjumlahan dua angka. Berdasarkan hasil program, seluruh konsep dasar Dart seperti variabel, percabangan, perulangan, list, dan fungsi sudah berjalan dengan baik. Output yang dihasilkan menampilkan data nama, umur, status kelulusan, hasil perulangan, data list, serta hasil penjumlahan.

Link Repository GitHub: https://github.com/Dsharlendra/Modul1-3GitFlutterDart_S1IF1104_2311102069_DsharlendraFebiandaAurelia